

SUMMARY

This topic covers AVL Trees and balanced search structures, emphasizing their properties, rotations, and mathematical relationships for maintaining efficient search performance.

AVL trees are self-balancing Binary Search Trees (BSTs) designed to prevent the worst-case $O(n)$ search time that occurs in skewed BSTs. By enforcing a Balance Factor (BF) constraint of -1, 0, or +1, AVL trees maintain a height of $O(\log n)$, ensuring consistent search, insertion, and deletion efficiency. Structural balance is restored through rotations, either via traditional LL, RR, LR, RL operations or the simplified "Triad" approach. Mathematical analysis provides formulas for maximum and minimum nodes at a given height, guiding both theoretical understanding and practical implementation.

1. LIMITATIONS OF BINARY SEARCH TREES (BST)

- Search efficiency depends on tree height (h).
- Balanced BST: Height $O(\log n) \rightarrow$ efficient searches.
- Skewed BST: Height $O(n) \rightarrow$ linear search time.
- Necessity of Balance: Maintaining $O(\log n)$ worst-case complexity motivates AVL tree design.

2. FUNDAMENTAL PROPERTIES OF AVL TREES

2.1 BST Property (Data-Related)

- Left Subtree < Node < Right Subtree
- Ensures search-ordering logic is maintained.

2.2 AVL Property (Structure-Related)

- Balance Factor (BF) = Height(Left Subtree) - Height(Right Subtree)
- BF must be -1, 0, or +1 for every node.
- Nodes outside this range (+2 or -2) indicate imbalance requiring rotation.

Property Type	Focus	Key Characteristic
BST Property	Data/Logic	Left < Node < Right
AVL Property	Structure/Height	$BF \in \{-1, 0, +1\}$

3. REBALANCING THROUGH ROTATIONS

Rotations restore AVL property after insertions or deletions.

3.1 Traditional Rotation Types

1. LL (Left-Left): Node inserted into left subtree of left child → single right rotation.
2. RR (Right-Right): Node inserted into right subtree of right child → single left rotation.
3. LR (Left-Right): Node inserted into right subtree of left child → double rotation.
4. RL (Right-Left): Node inserted into left subtree of right child → double rotation.

3.2 The "Triad Structure" Shortcut

- Identify the three nodes causing imbalance (critical node + two leading nodes).
- Sort keys in ascending order.
- Assign middle key as new root of substructure.
- Assign smallest key as left child and largest key as right child.

4. MATHEMATICAL ANALYSIS OF AVL TREES

4.1 Maximum Number of Nodes

- $N_{\max} = 2^{\{h+1\}} - 1$ (occurs in Full Binary Tree of height h).

4.2 Minimum Number of Nodes

- Recursive formula: $N(h) = 1 + N(h-1) + N(h-2)$
- Base Cases: $N(0) = 1, N(1) = 2$

Minimum Nodes by Height (h):

Height (h)	Minimum Nodes $N(h)$	Calculation
0	1	Base case
1	2	Base case
2	4	$1 + 2 + 1$
3	7	$1 + 4 + 2$
4	12	$1 + 7 + 4$
5	20	$1 + 12 + 7$
6	33	$1 + 20 + 12$
7	54	$1 + 33 + 20$
8	88	$1 + 54 + 33$

5. KEY PROCEDURAL INSIGHTS

- Insertion: Follows standard BST logic; AVL rebalancing occurs post-insertion.

- Critical Node Identification: The first node upward from the new key violating $BF \in \{-1, 0, +1\}$.
- Pointer Management: During rotations, subtrees must be reassigned to maintain BST ordering.

